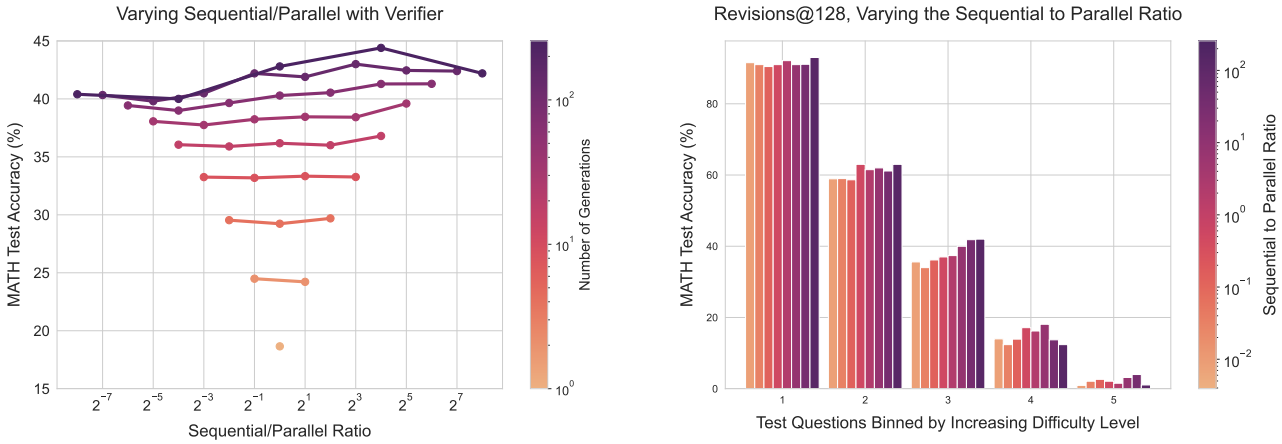

Figure 7 | **Left:** ***Varying the ratio of the generation budget allocated sequential revisions to verses parallel samples.*** Each line represents a fixed generation budget as the ratio is changed. We use the verifier for answer selection. We see that while increased sequential revisions tends to outperform more parallel compute, at higher generation budgets there is an ideal ratio that strikes a balance between the two extremes. **Right:** ***Varying the sequential to parallel ratio for a generation budget of 128 across difficulty bins.*** Using verifier-based selection, we see that the easier questions attain the best performance with full sequential compute. On the harder questions, there is an ideal ratio of sequential to parallel test-time compute.

achieves the maximum accuracy. We also see in Figure 7 (right) that the ideal ratio of sequential to parallel varies depending on a given question's difficulty. In particular, easy questions benefit more from sequential revisions, whereas on difficult questions it is optimal to strike a balance between sequential and parallel computation. This finding supports the hypothesis that sequential revisions (i.e., varying the proposal distribution) and parallel sampling (i.e., search with verifiers) are two complementary axes for scaling up test-time compute, which may be more effective on a per-prompt basis. We include examples of our model's generations in Appendix L. Additional results are shown in Appendix B.

Compute-optimal revisions. Given that the efficacy of sequential and parallel sampling depends on question difficulty, we can select the ideal ratio of sequential to parallel compute per difficulty bin. In Figure 8, we plot results using this compute-optimal scaling strategy when employing both our oracle and predicted notions of difficulty. In both cases, we're able to substantially improve test-time compute scaling by improving the proposal distribution via revisions. In particular, we see that at higher generation budgets, parallel sampling seems to plateau, whereas compute-optimal scaling demonstrates continued improvements. For both oracle and predicted difficulty bins, we see that **compute-optimal scaling can outperform best-of-N using up to *4x* less test-time compute** (e.g. 64 samples verses 256). Overall, these results demonstrate the potential for improved test-time compute scaling by adjusting the proposal distribution on a per-prompt basis.

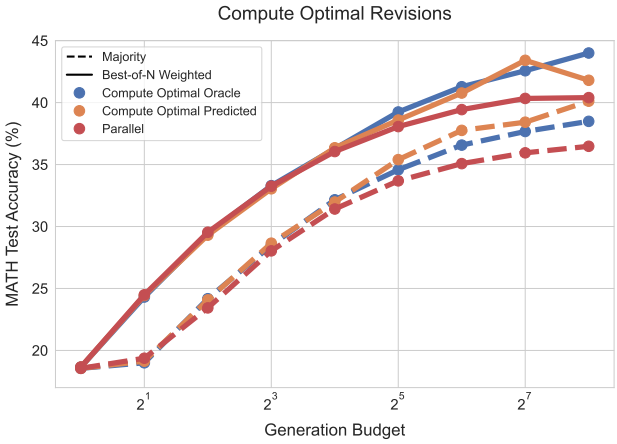

Figure 8 | ***Comparing compute-optimal test-time compute allocation against the parallel compute baseline with our revision model***. By optimally scaling test-time compute according to question difficulty, we find that we can outperform best-of-N using up to ***4x*** less test-time compute (e.g. 64 samples verses 256). "**Compute-optimal oracle**" refers to using the oracle difficulty bins derived from the ground truth correctness information, and "**compute optimal predicted**" refers to using the PRM's predictions to produce model-predicted difficulty bins.

Takeaways for compute-optimal scaling by refining the proposal distribution with revisions

We find that there exists a tradeoff between sequential (e.g. revisions) and parallel (e.g. standard best-of-N) test-time computation, and the ideal ratio of sequential to parallel test-time compute depends critially on both the compute budget and the specific question at hand. Specifically, easier questions benefit from purely sequential test-time compute, whereas harder questions often perform best with some ideal ratio of sequential to parallel compute. Moreover, by optimally selecting the best setting for a given question difficulty and test-time compute budget, we can outperform the parallel best-of-N baseline using up to *4x* less test-time compute.

7. Putting it Together: Exchanging Pretraining and Test-Time Compute

So far, we saw that utilizing additional test-time computation can enable us to represent more complex distributions than the one predicted by the base LLM itself, thereby improving performance. We now posit that this increased flexibility of representing distributions means that we can expect additional test-time compute to make up for the lack of a higher-capacity model or training for more FLOPs during pre-training. In this section, ***we study to what extent this is possible.*** We pose the following question:

Question: Exchanging pretraining and test-time compute

Suppose a model was pre-trained with X FLOPs. Assume that we plan to run Y FLOPs of inference with this model. If we want to improve performance by increasing the total FLOPs budget by a factor of M (i.e., $M(X+Y)$ total FLOPs across both pretraining and inference), should we spend our FLOPs on increased pretraining compute or on additional test-time compute?

Increasing pretraining FLOPS introduces the additional design decision of whether to allocate compute to training with more data or more parameters [14]. We focus on the setting in which model parameters are scaled up and training data amount is fixed, matching the approach taken with the open-source LLaMA series of models [41]. We choose this setting as it is representative of a canonical approach to scaling pretraining compute and leave the analysis of compute-optimal scaling of pretraining compute [29] where the data and parameters are both scaled equally to future work.

Defining an exchange rate between FLOPs. We now describe how we define the exchange rate between pretraining and inference FLOPs. To determine pretraining FLOPs, use use the common approximation $X = 6ND_{\text{pretrain}}$ [14], and for inference FLOPs, we use $Y = 2ND_{\text{inference}}$ [29]. Here N represents model parameters, D_{pretrain} is the number of tokens used for pretraining, and $D_{\text{inference}}$ the total number of tokens generated at inference time. With these approximations, we can see that, if we multiply the model parameters by a factor of M, then both the pretraining and inference FLOPs (due to the cost of greedy decoding with the larger model), increase by a factor of M (giving $M(X+Y)$ total FLOPs).

To match the FLOPs from scaling up model paramters using test-time compute with the smaller model we can multiply the smaller model's inference compute by a factor of $M + 3\left(\frac{D_{\text{pretrain}}}{D_{\text{inference}}}\right)(M-1)$. Notably, the amount of inference compute we can utilize to match the FLOPs for the larger model depends on the ratio $\frac{D_{\text{pretrain}}}{D_{\text{inference}}}$. We refer to the inverse of this ratio as R (e.g. $\frac{D_{\text{inference}}}{D_{\text{pretrain}}}$). Depending on the specific production setting or use-case, we should expect very different values of R. In particular, in many large scale production settings, we may expect significantly more inference tokens than pretraining tokens, in which case we would have $R >> 1$. On the other hand, in many contemporary self-improvement setups, that would use test-time compute to improve the model, we would likely generate significantly fewer

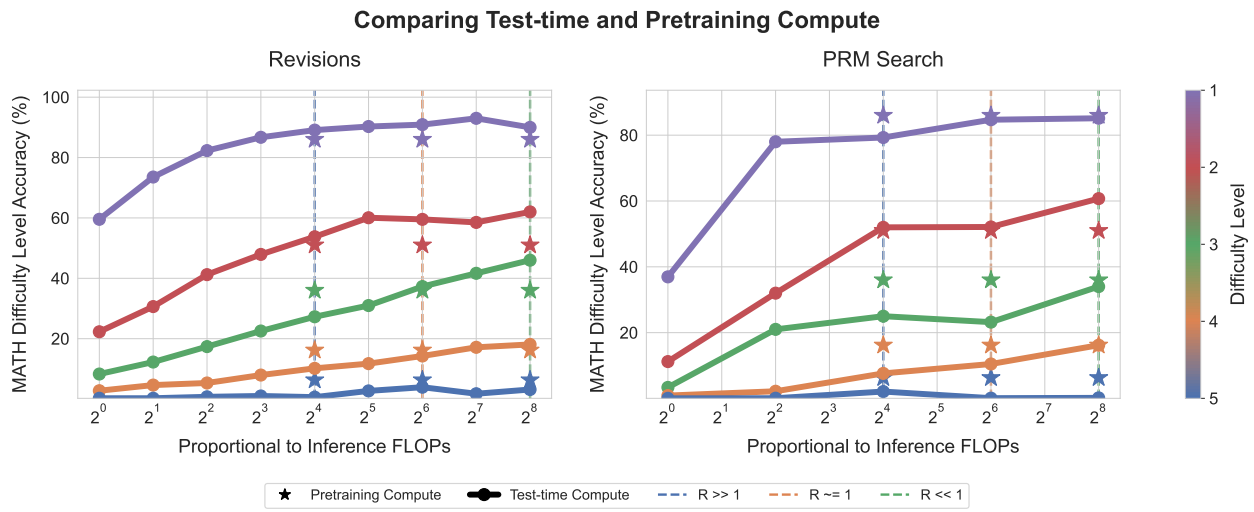

Figure 9 | ***Tradeoff between pretraining and test-time compute in a FLOPs-matched evaluation.*** Each line represents the performance of scaling test-time compute with our compute-optimal policy in each oracle difficulty bin. We plot the results for revisions on the left and search on the right. The stars represent the greedy pass@1 performance of a base model pretrained with ~ 14 times more parameters. We plot test-time compute budget on the x-axis, and place the stars at three different locations along the x-axis, each corresponding to the FLOPs equivalent point of comparison between scaling parameters and scaling test-time compute for three different inference compute loads (e.g. $R = \frac{D_{\text{inference}}}{D_{\text{pretrain}}}$). If the star is below the line, this implies that it is more effective to use test-time compute than to scale model parameters, and if the star is above the line this implies that scaling parameters is more effective. We see that on the easy questions or in settings with a lower inference load (e.g. $R << 1$), test-time compute can generally outperform scaling model parameters. However, on the harder questions or in settings with a higher inference load (e.g. $R >> 1$), pretraining is a more effective way to improve performance.

inference tokens than pretraining tokens, giving $R << 1$. Therefore, since the scale of test-time compute we can apply is dependent on this ratio, we expect differing conclusions depending on the specific setting.

In Figure 9, we use this approach to exchanging test-time and pretraining compute to compare our compute-optimal scaling against scaling up model parameters by a factor of ~ 14. We conduct comparisons for 3 different values of R: 0.16 ($R << 1$), 0.79 ($R \sim 1$), and 22 ($R >> 1$), with each ratio corresponding to an inference budget. Observe that if we only expect to see very difficult questions (e.g. difficulty bins 4/5) or have a larger $D_{\text{inference}}$ (corresponding to a larger R value), then it is often more effective to allocate our budget towards pretraining (e.g. the star is above the line). If instead, we expect mostly easy or intermediate difficulty questions (e.g. bins 1/2/3 and sometimes 4) or have lower inference requirements (as is the case in self-improvement pipelines), then utilizing test-time compute is better.

Takeaways for exchanging pretraining and test-time compute

Test-time and pretraining compute are not 1-to-1 "exchangeable". On easy and medium questions, which are within a model's capabilities, or in settings with small inference requirement, test-time compute can easily cover up for additional pretraining. However, on challenging questions which are outside a given base model's capabilities or under higher inference requirement, pretraining is likely more effective for improving performance.

8. Discussion and Future Work

In this work, we conducted a thorough analysis of the efficacy of different techniques that aim to either improve search against a verifier or to refine an LLM's proposal distribution, for scaling test-time compute